

LifeLog デプロイ・外部サービス連携設定手順書

本手順書では、LifeLogシステムを Google Cloud Platform (GCP) にデプロイし、Slack および Googleカレンダーと連携させるための詳細な手順について説明します。

1. 事前準備

1-1. ツール類のインストール

以下のツールがローカル環境にインストールされ、パスが通っていることを確認してください。

- [Google Cloud CLI \(gcloud\)](#)
- [Terraform](#)

1-2. Google Cloud プロジェクトの準備と認証

1. GCPコンソールにログインし、デプロイ対象のプロジェクトを作成します。
2. ローカルターミナルからGCPへの認証を行います。

```
``bash

gcloud auth login

gcloud auth application-default login

...
```

3. プロジェクトIDを設定します。

```
``bash

gcloud config set project [YOUR_PROJECT_ID]

...
```

2. Terraform によるインフラ自動構築

インフラ構成（Cloud Run, Firestore, BigQuery, IAMなど）をTerraformを使用してプロビジョニングします。

1. `terraform` ディレクトリに移動します。
2. `terraform.tfvars` ファイルを新規作成し、以下の必要な変数を定義します。

```
``hcl

project_id = "your-gcp-project-id"

region = "asia-northeast1"

...
```

3. 初期化およびインフラのデプロイを実行します。

```
``bash

terraform init

terraform apply

...
```

* ※ 実行完了後、ターミナルに表示される以下の出力結果（`outputs`）を記録してください：

- * `cloud_run_url` (Cloud Run Web APIのURL)
- * `github_actions_sa_email` (GitHub Actions用のサービスアカウント名)
- * `workload_identity_provider` (Workload Identityプロバイダ文字列)

Vertex AI API (Gemini API) の自動有効化について

Gemini APIとの通信に使用する **Vertex AI API** (`aiplatform.googleapis.com`) は、上記の `terraform apply` 実行時に自動的に有効化されます。Gemini自体は完全なサーバーレスのマネージドサービスであるため、インフラのプロビジョニング（サーバーやインスタンスの作成）は不要で、APIの有効化だけでそのまま利用可能です。

3. 外部サービス（Slack App）の設定

3-1. Slack アプリの作成

1. [Slack API Portal](#) にアクセスし、**Create New App** をクリックします。
2. **From scratch** を選択し、アプリ名（例: `LifeLog`）とワークスペースを設定します。

3-2. Bot Token Scopes (権限) の設定

1. メニューの **OAuth & Permissions** に移動します。
2. **Scopes > Bot Token Scopes** セクションで、以下のスコープを追加します：
 - * `chat:write` (Botからのメッセージ送信)
 - * `commands` (スラッシュコマンドの利用)
 - * `im:write` (DMでのユーザー会話起動)
3. **Install to Workspace** をクリックしてワークスペースにインストールし、発行された **Bot User OAuth Token** (`xoxb-...`) をコピーして保管します。

3-3. Signing Secret の取得

1. メニュー of **Basic Information** に移動します。
2. **App Credentials** セクションにある **Signing Secret** をコピーして保管します。

3-4. イベント購読（Event Subscriptions）の設定

1. メニューの **Event Subscriptions** に移動し、機能を **Enable** にします。
2. **Request URL** に以下を設定します：
 - * `[Cloud Run Web APIのURL]/api/slack/events`
3. **Subscribe to bot events** セクションで以下を追加します：
 - * `message.im` (DMでのメッセージ受信)
4. **Save Changes** をクリックします。

3-5. インタラクティブコンポーネント（Interactivity）の設定

1. メニューの **Interactivity & Shortcuts** に移動し、機能を **Enable** にします。
2. **Request URL** に以下を設定します：
 - * `[Cloud Run Web APIのURL]/api/slack/events`
3. **Save Changes** をクリックします。

4. Secret Manager への機密情報設定

Terraformによって作成された GCP Secret Manager に、Slack 関連の機密情報を登録します。

1. GCPコンソールの **Secret Manager** を開きます。
2. 以下のシークレットの新しいバージョンとして、先ほどコピーした値を登録します：
 - * `slack-bot-token` : **Bot User OAuth Token** (`xoxb-...`) を入力
 - * `slack-signing-secret` : **Signing Secret** を入力

5. 外部サービス（Google カレンダー）の連携設定

本アプリは、ランタイムのサービスアカウントを使用してユーザーのカレンダーにアクセスし、予定の自動取得・スケジュール登録を行います。最も安全でシンプルな「共有」方法を推奨しま

す。

5-1. サービスアカウントのメールアドレス確認

- アプリ実行用サービスアカウントのメールアドレスを確認します（通常 `lifelog-run-sa@[GCPプロジェクトID].iam.gserviceaccount.com` ）。

5-2. ユーザーカレンダーとの共有設定

1. 対象ユーザーの Googleカレンダーにアクセスします。
2. 連携したいカレンダー（「マイカレンダー」から選択）の右側の「:」をクリックし、**設定と共有** を開きます。
3. **特定のユーザーまたはグループとの共有** セクションで **ユーザーを追加** をクリックします。
4. **サービスアカウントのメールアドレス** を入力し、権限を **「予定の変更」** （または「予定の作成と変更」）に設定します。
5. **送信** をクリックして共有します。

6. アプリケーションのビルドとデプロイ

手動でのデプロイ手順

CI/CD を使用せずに手動でデプロイする場合は、GCP プロジェクトルートにて `gcloud builds submit` コマンドを使用してビルドを実行します。

1. Web API のビルド・デプロイ:

```
```bash

gcloud builds submit . \

--config cloudbuild-webapi.yaml \

--project [GCP_PROJECT_ID] \

--substitutions="_TAG=latest"

```
```

2. バッチジョブ（Cloud Run Jobs）のビルド・更新:

```
```bash

gcloud builds submit . \

--config cloudbuild-batch.yaml \

--project [GCP_PROJECT_ID] \

--substitutions="_TAG=latest"

```
```

GitHub Actions による自動デプロイ手順

特定のタグプッシュによって自動的にデプロイを行います。

1. GitHub Secrets の設定:

GitHub リポジトリの **Settings** > **Secrets and variables** > **Actions** にて以下のシークレットを登録します：

- * `GCP_PROJECT_ID` : デプロイ対象のGCPプロジェクトID
- * `WIF_PROVIDER` : Terraformの出力結果 `workload_identity_provider` の値
- * `WIF_SERVICE_ACCOUNT` : Terraformの出力結果 `github_actions_sa_email` の値

1. タグのプッシュによるCD実行:

* Web API のデプロイ:

`v-webapi-` で始まるタグをプッシュします。

```
```bash

git tag v-webapi-1.0.0

git push origin v-webapi-1.0.0

```
```

* バッチジョブのデプロイ:

v-batch- で始まるタグをプッシュします。

```
``bash
```

```
git tag v-batch-1.0.0
```

```
git push origin v-batch-1.0.0
```

```
``
```